

AlpacaAgent desk reference

Commands, controls, troubleshooting, and recovery

A practical reference for the current Windows implementation, with explicit command effects and evidence-based incident procedures.

DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

1. Quick command chooser	3
2. Launch modes and controls	4
3. Research and scanner commands	5
4. Diagnose startup, dashboard, and chat problems	6
5. Diagnose reconciliation and data failures	7
6. Ambiguous submissions and intent recovery	8
7. Manual flatten and stopping safely	9
8. History gaps, storage, locks, and backups	10
9. File map, metrics, and glossary	11
10. Corrections to older notes and source map	12
11. External sources and final readiness card	13

1. Quick command chooser

Reviewed against repository revision 235b087 on September 18, 2026. Run commands from C:\Users\eveli\OneDrive\Desktop\AlpacaAgent unless you deliberately use another checkout. Examples use python; substitute your working Python 3.11+ invocation if necessary.

Intent	Command or entry point	Effect
Open saved dashboard	Open-Dashboard.cmd	Rebuilds local HTML; opens browser
Generate dashboard	python -m alpaca_agents.dashboard	Reads local records; writes HTML
Serve local workspace	python -m alpaca_agents.controller --serve	Starts server; no periodic cycles
Query broker account	python -m alpaca_agents.executor account	Broker GET and request trace
Reconcile state	python -m alpaca_agents.executor reconcile	Broker reads; imports/updates local evidence
Read request traces	python -m alpaca_agents.executor traces	Local trace-store inspection
Inspect live intents	python -m alpaca_agents.executor intents	Journal read plus broker account access
Run one controller cycle	python -m alpaca_agents.controller	No POST; can update local state
Run repeated cycles	controller --every 300	Requires full module prefix; no POST without --submit
Run shadow scan	scanner --session DATE	Requires full module prefix; no broker orders
Replay cached history	backtest --bars-file FILE --symbol QQQ	Requires full module prefix; local research

Short module names in the last three rows mean `python -m alpaca_agents.<module>`. Full copyable examples appear in the following chapters. "No POST" does not mean "no disk writes": diagnostics, activity imports, cycle reports, and some dry reservations change local evidence.

Most important operational distinctions

Reconciled does not mean authorized. Authorized does not mean submitted. Submitted does not mean filled. A fill does not mean a lifecycle is fully closed. A recently rendered page does not mean the broker was recently queried. A backtest result does not mean option P&L was simulated.

Exit statuses

executor reconcile returns 0 for reconciled state and 2 for unreconciled state; operational failures generally return 1. A single controller cycle returns 0 when reconciliation is okay and 2 when it is not, even if later stages need closer inspection. The terminal loop uses 3 for designated state incidents, and 0 for its ordinary market-closed termination. Always read the report; exit status alone does not describe every stage.

2. Launch modes and controls

Static, served, and scheduled

```
python -m alpaca_agents.dashboard
python -m alpaca_agents.controller --runtime runtime --serve
python -m alpaca_agents.controller --runtime runtime --every 300 --dashboard
python -m alpaca_agents.controller --runtime runtime --serve --every 300
```

These are alternatives, not four commands to run simultaneously. One controller owns a runtime directory. --every must be at least 60 seconds. --serve --submit requires --every because on-demand chat diagnostics are always dry. Scheduled work in a served process may submit if the process was separately launched with all trading gates; chat itself still cannot submit.

The control file

Exact trimmed text	New entries	Exit handling
ARMED_PAPER	Eligible if all other gates pass	Eligible if other conditions pass
EXITS_ONLY	Blocked	Eligible if other conditions pass
DISABLED or anything invalid	Blocked	Blocked

No mode creates a process, supplies credentials, guarantees a fill, cancels an existing order, or clears an incident. Automatic submission still requires --submit. The control file is reread during decisions; playbook launch configuration is loaded when the controller initializes.

```
# Choose ONE intentional mode; these overwrite the file.
@'
from pathlib import Path
Path('runtime/trading-control').write_text('EXITS_ONLY', encoding='utf-8')
'@ | python
```

For a complete stop, substitute DISABLED after considering any positions and outstanding orders. For an approved paper pilot, substitute ARMED_PAPER. Direct marker writes are appropriate for a stopped controller; an operational process should use an atomic replacement procedure to avoid a partially written file. Invalid or incomplete reads fail closed.

Approvals are separate

The marker runtime/playbooks/trend_directional.approved must contain APPROVED, and the launch must include --enable-playbook trend_directional. Repeat --enable-playbook for multiple reviewed playbooks. Do not enable multiple playbooks during your first experiment just because the syntax permits it.

If using --runtime elsewhere, keep every executor database and control path aligned. Executor commands do not share the controller's --runtime flag: they expose individual --fills-db, --orders-db, --history-db, --trace-db, and --control-file arguments. Mixing directories can create misleading or account-inconsistent evidence.

3. Research and scanner commands

Fetch historical bars and report

```
python -m alpaca_agents.backtest --symbol QQQ `
--start 2023-01-01 --end 2026-09-17 `
--save-bars runtime/bars-QQQ.json `
--playbooks trend oversold_bounce breakout `
--output runtime/bt-QQQ.json
```

Both dates are required for a network run. This is an example window, not a permanently current command. Use an end date whose bars are complete. The market-data key belongs in the process environment.

```
python -m alpaca_agents.backtest --symbol QQQ `
--bars-file runtime/bars-QQQ.json `
--playbooks trend --output runtime/bt-QQQ.json
```

The cached run does not refetch history. It overwrites that output with the selected playbook report, so archive any multi-playbook report you want to retain. The dashboard discovers bt-*.json; the generative symbol tool expects bt-<SYMBOL>.json. CLI default backtest-<SYMBOL>.json is a different filename.

Standalone shadow scan

```
python -m alpaca_agents.scanner --session 2026-09-17 `
--symbols SPY QQQ IWM DIA `
--output runtime/shadow-scan.json
```

Set --session to the last completed exchange session. The option quotes are current, not historical quotes from that date. Default symbols are SPY QQQ IWM; DIA is optional. --audit can override runtime/market-data.jsonl, but audit and output must not be the same file.

Read reports without changing trading permissions

```
Get-Content runtime\shadow-scan.json -Raw | ConvertFrom-Json |
  ConvertTo-Json -Depth 12
Get-Content runtime\bt-QQQ.json -Raw | ConvertFrom-Json |
  Select-Object symbol, first_bar, last_bar, options_pnl_modelled, summary
```

The second command is a compact summary; inspect the full summary/trades objects for detailed research. Preserve source bars, exact command, code revision, and report filename for each important comparison.

Do not confuse these names

Backtest trend maps conceptually to trend-based scanner playbooks; it does not model a call versus a debit spread. Backtest breakout is not the launch string breakout_continuation. The current executable single-leg choices are trend_directional and oversold_bounce, subject to actual candidate structures and all other checks. The existence of a playbook name is not a recommendation to enable it.

4. Diagnose startup, dashboard, and chat problems

"python is not recognized" or "No module named alpaca_agents"

For a missing command, verify the Python installation and PATH or use `py -3` if available. For a missing module, install the project into the same interpreter from the project root using the command below. Its final dot denotes the current directory.

```
python -m pip install -e .  
python -m alpaca_agents.controller --help
```

Dashboard looks stale

Static mode requires regenerating HTML after runtime records change. Served questions reread local records, but main panels need a page refresh. Check the latest cycle timestamp, not only the rendered time. A browser connection does not prove the scheduled worker is still cycling.

Chat says it is rule-based

That is expected in static mode and in served mode without a valid `ANTHROPIC_API_KEY`. Restart the server after changing its environment. A model/provider failure or exhausted budget can also cause fallback. Read the answer's source label. Nova is generative-only.

Chat cannot answer a detailed file question

The model has a small allowlist of tools, not arbitrary filesystem access. It can read defined projections of snapshots, backtests, and cycles, plus rule explanations and optional market/news data. Open the actual JSON if your analysis needs fields the tool omits. Repeatedly asking does not grant the missing tool.

Houston did not run a diagnostic

In served mode, select Houston and send exactly `reconcile`. The server trims whitespace and compares case-insensitively. In static mode it cannot run a diagnostic. A sentence such as "reconcile and place this trade" is not the special command and cannot place an order.

Local URL no longer works

Keep the owning terminal open and use the URL printed by the current launch. The port and session token are launch-specific. Restarting the server can invalidate an old tab. The server is loopback-only; exposing it to another machine is not a supported operator shortcut.

Budget unexpectedly consumed

The generative cap counts provider calls, including tool rounds, not just questions. Usage is held in memory and resets with the process. It is not a durable billing ceiling. Check the provider's usage controls if you need a lasting financial limit.

5. Diagnose reconciliation and data failures

Work from exact reasons

```
python -m alpaca_agents.executor reconcile
python -m alpaca_agents.executor traces
```

Record the timestamp, all reasons, and relevant request identifiers. Do not replace an exact error with an informal label in your incident notes. Several reasons can occur together; fixing one does not establish full reconciliation.

Reason or symptom	Interpretation	First action
MARKET_CLOSED / NOT_A_SESSION	Trading-session gate	Check session context; inspect other reasons
BROKER_CLOCK_SKEW / STALE_STATE	Time evidence too old or inconsistent	Check system time and request latency
ACCOUNT_*	Account flags, eligibility, or status failed	Inspect current paper-account settings
NOT_CASH_ACCOUNT	Cash/margin policy not satisfied	Review account type and acknowledgement policy
HISTORY_*	Coverage, recency, or anchoring problem	Preserve import evidence; inspect exact suffix
POSITION_MISMATCH	Broker and local inventory disagree	Compare fills, activities, and positions
UNKNOWN_OPEN_ORDER	Order lacks expected local provenance	Inspect broker client order ID and journal
CLAIMED_INTENT_WITHOUT_BROKER_RECORD	Claimed intent lacks matching broker record	Treat as an ambiguous submission incident
HTTP 403 from option snapshots	Request not authorized for that data	Check credentials and exact data entitlement
Quote freshness rejection	Quote too old or invalid for valuation time	Inspect timestamps and provider response

Broker readiness and data readiness are independent

A successful account GET does not prove settled cash. A reconciled broker state does not prove options-data access. Stock bars working does not prove option snapshots work. A valid option snapshot does not prove every contract has complete eligible Greeks, interest, prices, or timestamps.

Quote age is capped at 120 seconds in the current snapshot adapter. The risk-state freshness window is 60 seconds. History coverage has its own fifteen-minute lag allowance. These are different clocks. Do not fix one by changing the others.

What to preserve

Keep cycles.jsonl, notifications.jsonl, market-data.jsonl, api-requests.sqlite3, activities.sqlite3, fills.sqlite3, and orders.sqlite3 together. The source-specific logs explain different parts of an incident. Include a sanitized excerpt and relevant IDs when reporting a bug; exclude credentials, full environment dumps, and unnecessary account data.

6. Ambiguous submissions and intent recovery

Why an unknown result is special

The broker may have accepted an order even if the client lost the response. The journal intentionally keeps an unknown submission claimed. The reserved capacity remains relevant until reconciliation or a controlled operator action resolves the situation. An HTTP failure is not automatically evidence that no order exists.

Investigation sequence

1. Stop new entries using the appropriate control mode; consider held positions before disabling exits.
2. Read the latest cycle, notifications, intents, and broker request traces.
3. Identify the authorization_id and client_order_id; do not guess from a ticker alone.
4. Inspect the paper broker's order history for that exact order and any fills.
5. Run reconciliation and review whether terminal broker records can be resolved by a controller diagnostic.
6. Use release-intent only when its specific prerequisites are met and your operator note documents the verification.

```
python -m alpaca_agents.executor intents
python -m alpaca_agents.executor traces
python -m alpaca_agents.executor reconcile
```

The intents command also needs paper credentials because it opens the account-bound journal after fetching the broker account. traces is the local inspection command.

Controlled release

The following is syntax, not an instruction to release an unidentified intent. Replace the placeholder with the exact authorization ID and write a factual note.

```
python -m alpaca_agents.executor release-intent AUTHORIZATION_ID `
--note "Verified exact client order ID in paper broker history at TIME; findings recorded in incident notes"
```

The command performs its own broker lookup. If it finds a terminal order, it can resolve from that status. If the order is still live, it refuses release. If the journal already has a broker acknowledgement, absence from one lookup is not sufficient for release. For an eligible unacknowledged claim, a broker not-found trace and operator note support the audited release.

Never use these recovery shortcuts

Do not submit the same intended trade again to see whether it works. Do not delete the order database, manually change claimed to rejected, remove a broker ID, or clear a loss latch. These actions destroy the evidence needed to prevent duplicate exposure and reconcile later fills.

Completion criterion

The broker's order/fill state and the journal agree, the incident is documented, and a fresh cycle has no unexplained state reasons. If uncertainty remains, leave it unresolved and obtain a code-level investigation rather than inventing a clean state.

7. Manual flatten and stopping safely

What flatten actually does

The manual command finds held quantity in the local fill ledger, reads the broker mark, creates an audited exit through the existing journal, and prices a day-limit sell at 95% of the mark with a \$0.01 floor. It does not retrieve the automatic manager's fresh NBBO bid. It is not an instant liquidation guarantee.

The manual implementation does not perform the controller's complete reconciliation sequence before preparing the exit. Run diagnostics first and confirm the exact held contract, quantity, account, and outstanding orders. The journal still applies its exit validation and control checks.

Preview first

Use the exact OCC contract copied from verified position evidence. OCC_CONTRACT below is a placeholder, not an instrument to trade.

```
python -m alpaca_agents.executor flatten OCC_CONTRACT
```

A successful dry preview prepares and then releases an unsent exit in the journal. It writes audit evidence; it is not a purely read-only display. Inspect contract, side, quantity, and limit. If the command says it cannot price or the ledger does not hold the contract, investigate rather than forcing the order.

Submit only when you intend the paper exit

```
python -m alpaca_agents.executor flatten OCC_CONTRACT --submit
```

Verify the returned outcome at the broker and reconcile after any fills. Do not rerun it after an ambiguous result. The command does not implement a general cancel/replace workflow for an existing unfilled order.

Pausing entries versus stopping all automation

For winding down, set EXITS_ONLY and keep a correctly configured submission-enabled controller running. For a complete automation stop, use DISABLED only after deciding how any open positions and pending orders will be handled. Both settings affect future local decisions; neither cancels broker orders already accepted.

If manual broker action becomes necessary to manage a position, preserve exact order/fill evidence and expect reconciliation to require attention. The normal workflow relies on orders originating in the local journal; unexplained external orders violate that provenance assumption.

Before closing the terminal

Confirm: positions flat or explicitly managed; pending broker orders understood; ambiguous intents resolved or explicitly escalated; runtime evidence preserved; next session's restart plan known. Do not use "dashboard shows no new notification" as evidence that the broker is flat.

8. History gaps, storage, locks, and backups

Activity imports require explicit bounds

Older notes show import-activities without its required arguments. The CLI requires --after and --until, using timezone-aware timestamps. Select bounds from the diagnosed gap and account history; do not copy a random range and assume completeness.

```
# Example syntax only: replace these bounds with the diagnosed window.
python -m alpaca_agents.executor import-activities `
  --after 2026-09-16T00:00:00+00:00 `
  --until 2026-09-18T12:00:00+00:00 --max-pages 100
python -m alpaca_agents.executor reconcile
```

Exhausting pages is not by itself verified history completeness. The reconciler requires anchored, continuous coverage from account creation and sufficient recency. Provider time bounds are exclusive, so merely touching import windows can leave a gap; use the code's overlap-aware coverage model. Never fabricate earlier fills to make inventory match.

Stale controller lock

```
Get-Content runtime\controller.lock
```

The file records a process ID and timestamp. Confirm that the specific old controller has terminated, including checking whether an ID has been reused for another process. Only after that verification, remove that exact stale file:

```
Remove-Item -LiteralPath .\runtime\controller.lock
```

Do not remove it to make room for a second running controller. A clean normal shutdown should remove the lock itself. The lock protects controller ownership, not every possible separate executor CLI invocation; avoid concurrent manual recovery commands while scheduled work is active.

Consistent backups

Stop active writers cleanly before making a filesystem backup of runtime. Preserve the directory as a unit, including any SQLite companion files present, reports, controls, approvals, and logs. Keep backups access-restricted because they contain account activity and operational evidence. The location under OneDrive does not make a concurrent copy a verified consistent SQLite backup.

Never restore an old runtime over active trading state simply to roll back an error. Broker orders and fills do not roll back with local files. A restored copy must be reconciled against current broker history before operation resumes. Keep one account per ledger/journal set.

Storage failures

If audit or persistence fails, the application stops the affected operation. Check disk space, permissions, locking, and filesystem availability. Preserve the error context. Clearing files to gain a clean dashboard can erase the very evidence needed to know whether an order was sent.

9. File map, metrics, and glossary

Runtime item	Purpose
trading-control	Exact local entry/exit mode
paper-margin-acknowledged	Explicit margin-paper policy acknowledgement
playbooks/*.approved	Reviewed playbook markers
controller.lock	Exclusive controller ownership marker
cycles.jsonl	One structured record per controller cycle
notifications.jsonl	Local warnings, incidents, and informational events
market-data.jsonl	Market-data request and filtering diagnostics
shadow-scan.json	Most recent standalone shadow report
bt-*.json	Dashboard-discovered underlying research reports
bars-*.json	Cached historical bars for repeatable research
api-requests.sqlite3	Broker request metadata and request identifiers
activities.sqlite3	Staged raw broker activities and import coverage
fills.sqlite3	Fill-level FIFO inventory, realizations, and loss latches
orders.sqlite3	Intent journal, stored bodies, and transitions
dashboard.html	Generated view; not the trading state authority

The separate TradeLedger/trades.sqlite3 prototype is not a second P&L source to add to FillLedger. Summing both can double-count realizations. Use the active fill-based accounting and understand its source coverage.

Key definitions

Basis: remaining acquisition cost, including allocated entry fees in the ledger. It differs from market value. **Mark:** broker-provided option price used as an observation, not a guaranteed executable price. **Bid/ask:** quoted buy/sell sides of the market; a limit still requires execution.

Reconciliation: comparison of broker account, positions, orders, clock, activities, and local evidence. **Reservation:** temporary capacity allocation. **Claim:** journal transition binding an intent to the prepared submission path. **client_order_id:** application-generated identifier used to correlate the broker order. **Request ID:** broker response identifier useful for support; it is not an order ID.

Mean R: average underlying outcome relative to modeled stop distance. **Median R:** middle outcome. **Profit factor:** aggregate positive R relative to absolute negative R, subject to zero-loss edge cases. **Win rate:** fraction of resolved outcomes with positive R. **Target-hit rate:** frequency of the target exit, not the same statistic as win rate.

no_fill: modeled trigger never produced an entry. **no_trade:** the modeled fill was already beyond a permitted stop/target boundary. **unresolved:** the dataset ended without a completed outcome. A report's **sample_sufficient** flag is a minimum-count check, not a proof of an edge.

10. Corrections to older notes and source map

Implementation details this guide deliberately corrects

- A DISABLED controller does not run the entry scanner. Use the standalone scanner for disabled shadow work.
- Network backtests require both --start and --end.
- import-activities requires --after and --until.
- Manual flatten uses 95% of broker mark, while automatic exits prefer a fresh bid when available.
- A dry enabled controller cycle can create a reservation; it is not entirely read-only.
- Default backtest filenames do not match dashboard bt-*.json discovery.
- Generative chat now has optional stock snapshot/news tools; descriptions saying it can only read local records are incomplete.
- Current trend entry plans have a 21-DTE time stop, not the replay's fifteen-session holding limit.
- Circuit-breaker decisions use CIRCUIT_BREAKER in the rules layer; do not expect the informal label BREAKER_TRIPPED everywhere.
- Cycle entries are nested under stages.entries.entries, and the scan summary is stages.scan. A generic halt_reason field is not the current cycle schema.

Local source map

All following paths are under `src/alpaca_agents/`.

`controller.py` defines CLI flags, runtime locking, cycle sequencing, approvals, and loop halts. `gateway.py` defines control-file interpretation. `rules.py` defines pure entry checks. `scanner/scan.py`, `signals.py`, and `contracts.py` define ideas, playbooks, exits plans, and structure filters. `marketdata/snapshot.py` defines snapshot normalization and quote-age checks.

`executor/reconcile.py` defines account, clock, history, inventory, and local cash/capital constraints. `executor/orders.py` defines reservations, claims, stored order bodies, exit preparation, and TTL. `executor/submit.py` implements single-attempt submission. `executor/exits.py` defines automatic exit evaluation. `executor/__main__.py` defines manual diagnostics, release-intent, and flatten.

`dashboard.py` defines report discovery and display projections. `dashboard_chat.py` and `assets/dashboard.js` define deterministic chat and browser behavior. `studio.py` defines the loopback server and exact Houston diagnostic request. `llm_chat.py` defines optional provider configuration, tools, usage counting, and fallback. `backtest/replay.py` and `__main__.py` define replay assumptions and report fields.

`README.md`, `PLAYBOOK.md`, and `HANDOVER.md` provide rationale and history, but contain superseded details. When upgrading the repository, compare these source locations before reusing operational commands from this dated guide.

11. External sources and final readiness card

Provider documentation

External pages were checked during preparation on September 18, 2026. They describe provider capabilities, which may be broader than this application's implementation. Provider pricing, permissions, supported products, and account rules can change; verify them before making account or subscription decisions.

[Alpaca paper-trading documentation](#) explains the simulated paper environment and its limitations. Paper execution is not equivalent to a demonstrated live execution record. The application pins its own paper endpoint; generic provider instructions for changing a base URL do not create a supported live configuration here.

[Alpaca options-trading documentation](#) describes brokerage options capabilities and order concepts. The broker may offer structures that this local journal refuses. Broker capability is not application implementation.

[Massive options API overview](#) describes option snapshots and market-data fields. [Massive options pricing](#) is the place to verify current subscription offerings. Confirm exact endpoint, quotes, and realtime access instead of relying on historical plan names in local notes.

This guide documents the code's internal risk and same-session policies. It does not assert that these implement every current legal, regulatory, or broker requirement. No provider fee schedule or subscription price is hardcoded into the recommendations here.

Before enabling a paper session

1. Correct runtime and paper account; current protected credentials.
2. Broker reconciliation understood and current for the session.
3. Required stock and options data available with eligible freshness.
4. One reviewed executable playbook and explicit symbol universe.
5. Matching approval marker and launch flag.
6. Intentional control mode and submission flag.
7. Known open positions and outstanding orders; no unexplained intent.
8. Machine and controller will remain available for the planned period.
9. Exit behavior and known research/execution gaps understood.
10. Session plan, incident procedure, and review record ready.

If anything looks wrong

Read the exact record, preserve the evidence, and identify the earliest failed condition. Pause new risk when appropriate. Keep existing positions explicitly managed. Do not clear databases, loosen checks, retry unknown submissions, or mistake a missing record for a zero value. Resume only after the relevant broker and local evidence agree.